Use two partners:

1. **Verizon listing crawler**
   Source: https://www.verizon.com/smartphones/
2. **T-Mobile PDP scraper**
   Source listing: https://www.t-mobile.com/cell-phones

Goal

Build a Python scraper that handles two different scraping patterns:

- From **Verizon**, start at the listing page, discover product cards/PDP links, and extract listing-level data.
- From **T-Mobile**, scrape 2 manually supplied PDP URLs and extract deeper product/variant/promo data.

Part A: Verizon Listing Crawler

Input:

python assignment.py verizon --listing-url https://www.verizon.com/smartphones/ --limit 8

Expected behavior:

- Load Verizon smartphone listing.
- Extract the first 8 real phone products.
    - Condition: Check if "iPhone 17" is included in these first 8 products. If not, explicitly search the listing page to find the iPhone 17 and extract its link.
- Handle lazy loading or pagination if needed.
- Deduplicate products.
- Interact with the “all offers” button for each product to extract active promotions and any “more savings” details, mapping these to a new field in the JSON.
- For each product, capture:

```
{
  "partner": "verizon",
  "source_type": "listing",
  "product_name": "...",
  "brand": "...",
  "listing_url": "...",
  "pdp_url": "...",
  "monthly_price": "...",
  "retail_price": "...",
```

```
  "visible_colours": ["..."],
  "availability_hint": "...",
  "offers": ["..."],
  "errors": []
}
```

Stretch requirement:

- Visit PDPs for the first 3 products and compare whether PDP price matches listing price.
- Add a `price_match_status`: `match`, `mismatch`, or `unknown`.

Part B: T-Mobile PDP Scraper

Input:

python assignment.py tmobile --urls tmobile_urls.txt

Tmobile links:

1. https://www.t-mobile.com/cell-phone/apple-iphone-16-plus
2. https://www.t-mobile.com/cell-phone/apple-iphone-17-pro-max

Expected behavior:

- Load each T-Mobile PDP.
- Extract product and commercial fields.
- Interact with at least two dynamic UI elements, for example:
    - storage selector,
    - colour selector,
    - promotions modal,
    - “see all promotions” section,
    - financing/full-price toggle if available.

Output per PDP:

```
{
  "partner": "tmobile",
  "source_type": "pdp",
  "source_url": "...",
  "product_name": "...",
  "brand": "...",
  "available_colours": ["..."],
```

```
  "available_storage_options": ["..."],
  "selected_colour": "...",
  "selected_storage": "...",
  "monthly_price": "...",
  "full_retail_price": "...",
  "promotions": [
    {
      "title": "...",
      "description": "...",
      "source_interaction": "promo_modal"
    }
  ],
  "availability_status": "...",
  "errors": []
}
```

Required Deliverables

- `assignment.py` or a small package with clear entrypoints.
- `requirements.txt`.
- `README.md`.
- `outputs/verizon_listing.json`.
- `outputs/tmobile_pdp.json`.
- `screenshots/` with at least one screenshot per scraped PDP/listing page.
- A short debugging note:
  “What broke, what selectors were unstable, and what fallback did you implement?”

Rules

- Selenium is allowed. Playwright is allowed but not required.
- No manual selector entry at runtime.
- No hard-coded final JSON. The script must actually run.
- No login, checkout, cart, or address flow required.
- Fixed sleeps are allowed only as a last resort; primary waiting should use explicit waits.
- If a field fails, output the record with `null` and add an error message rather than crashing the whole run.

Deadline

- Submit within 24 hours.
- It does not need to be perfect.

- We care more about approach, robustness, and debugging notes than 100% field coverage.
- Minimum acceptable completion: Verizon listing works for 8 products and T-Mobile works for at least 1 PDP with screenshot and partial fields.